

LAPORAN UTS INDIVIDU
COMMUNICATION PROTOCOL - CASE 1: USER/PROFILE API

Disusun Oleh:
Putri Sahira Wulandari
NIM: 25120500004
Prodi Sains Data - Kelas Profesional

Dosen Haikal Shiddiq, S.Kom., M.T
Universitas Cakrawala
Tahun 2026

DAFTAR ISI

BAB I PENDAHULUAN	3
1.1 Latar Belakang	3
1.2 Rumusan Masalah	3
1.3 Tujuan Praktikum	3
BAB II LANDASAN TEORI	4
2.1 REST API	4
2.2 HTTP Methods	4
2.3 HTTP Status Codes	4
2.4 Tools Postman	4
BAB III HASIL DAN PEMBAHASAN	5
3.1 Konfigurasi Awal	5
3.2 Request 1: GET /api/users	6
3.3 Request 2: GET /api/users/1	7
3.4 Request 3: POST /api/users	8
3.5 Request 4: PUT /api/users/1	9
3.6 Rangkuman Status Code	10
BAB IV KESIMPULAN DAN SARAN	11
4.1 Kesimpulan	11
4.2 Saran	11
4.3 Github	11
4.4 Wireshark	12
4.5 Google Slide	12
DAFTAR GAMBAR	
Gambar 1. Response GET /api/users	5
Gambar 2. Response GET /api/users/1	6
Gambar 3. Response POST /api/users	7
Gambar 4. Error 400 Invalid fields saat POST	7
Gambar 5. Response PUT /api/users/1	8
Gambar 6. Error 404 Not Found saat PUT	8

BAB I PENDAHULUAN

1.1 Latar Belakang

Application Programming Interface atau API menjadi tulang punggung komunikasi data pada aplikasi modern. Hampir semua layanan digital seperti Gojek, Tokopedia, dan aplikasi perbankan menggunakan REST API untuk menghubungkan frontend dengan backend. Protokol utama yang digunakan adalah HTTP dengan berbagai method seperti GET, POST, PUT, dan DELETE untuk menjalankan operasi CRUD.

Pemahaman terhadap cara kerja HTTP Method, Status Code, dan format JSON sangat penting bagi mahasiswa Sains Data. Kemampuan melakukan testing API menggunakan tools seperti Postman menjadi skill dasar sebelum masuk ke tahap integrasi data dan deployment model. Oleh karena itu, praktikum ini dilakukan untuk menguji endpoint `/api/users`` secara langsung.

1.2 Rumusan Masalah

1. Bagaimana implementasi HTTP Method GET, POST, dan PUT pada endpoint `/api/users` menggunakan Postman?
2. Apa makna dari Status Code 200 OK, 201 Created, 400 Bad Request, dan 404 Not Found yang muncul selama pengujian?
3. Bagaimana struktur request body dan response body JSON yang diterima oleh server?

1.3 Tujuan Praktikum

1. Melakukan minimal 2 request GET dan 2 request POST pada Case 1 sesuai instruksi UTS.
2. Menganalisis status code, headers, request body, dan response body dari setiap request.
3. Mendokumentasikan evidence berupa screenshot dan file export Collection Postman sebagai bukti pengerjaan.

BAB II LANDASAN TEORI

2.1 REST API

REST atau Representational State Transfer adalah gaya arsitektur untuk sistem terdistribusi. REST bersifat stateless, artinya server tidak menyimpan state client di antara request. Setiap resource diidentifikasi melalui URL, contoh `/api/users` untuk resource user.

2.2 HTTP Methods

Empat method utama yang digunakan pada praktikum ini adalah:

1. **GET**: Digunakan untuk mengambil data dari server. Bersifat safe dan idempotent. Tidak memiliki request body.
2. **POST**: Digunakan untuk membuat resource baru di server. Tidak idempotent. Memerlukan request body berisi data yang akan dibuat.
3. **PUT**: Digunakan untuk memperbarui resource secara keseluruhan. Bersifat idempotent. Memerlukan request body lengkap.
4. ***DELETE***: Digunakan untuk menghapus resource. Tidak digunakan pada Case 1 ini.

2.3 HTTP Status Codes

Status code menunjukkan hasil dari request yang dikirim ke server.

Kode	Kategori	Arti	Contoh Kasus di Pratikum
200 OK	2xx Success	Request berhasil diproses	GET <code>/api/users</code> berhasil
201 Created	2xx Success	Resource baru berhasil dibuat	POST <code>/api/users</code> berhasil
400	4xx Client Error	Request tidak valid	Body kurang field required
404 Not Found	4xx Client Error	Resource tidak ditemukan	PUT tanpa <code>/id</code> di URL

2.4 Tools Postman

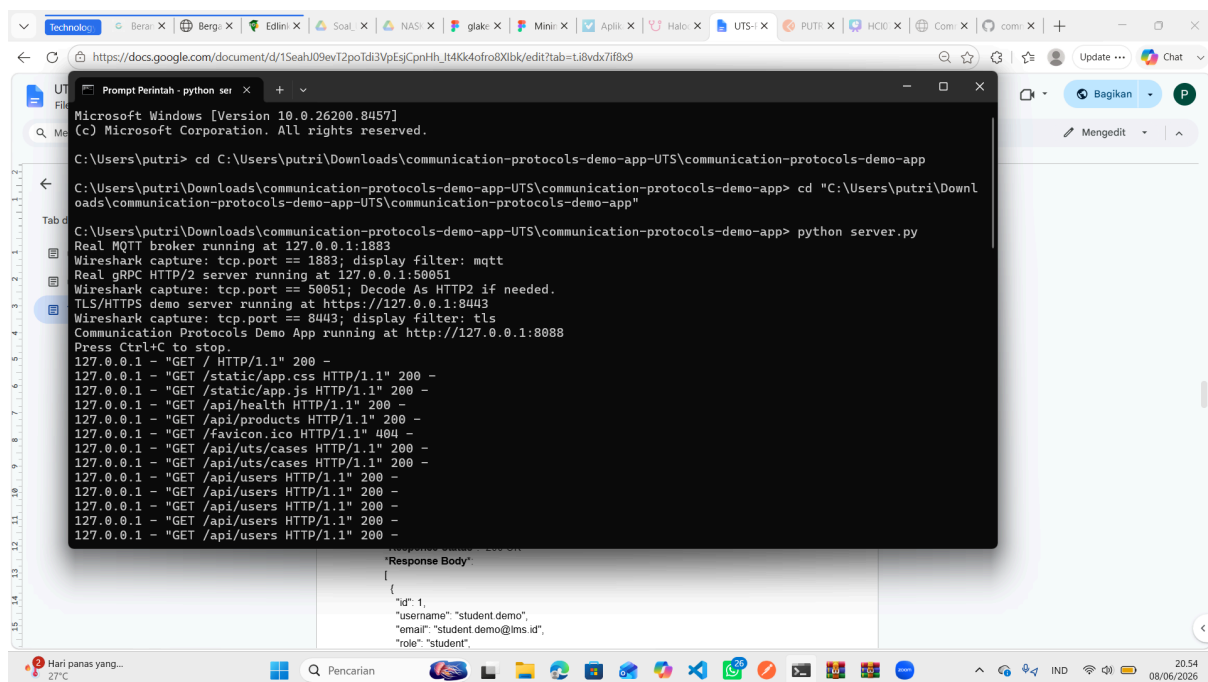
Postman adalah aplikasi untuk melakukan pengujian API. Fitur utama yang digunakan adalah Collections untuk mengelompokkan request, Environments untuk menyimpan variable seperti ``base_url``, serta tab Headers dan Body untuk mengatur request.

BAB III HASIL DAN PEMBAHASAN

3.1 Konfigurasi Awal

Tahap awal praktikum adalah instalasi server lokal untuk pengujian API Case 1. Langkah yang dilakukan:

1. **Download Source Code:** File [communication-protocols-demo-app-UTS.zip](https://github.com/haikalshiddiq/communication-protocols-demo-app-UTS) diunduh dari `https://github.com/haikalshiddiq/communication-protocols-demo-app-UTS`.
2. **Ekstrak & Install:** File di ekstrak menggunakan WinRAR. Aplikasi dijalankan dengan Python melalui perintah `python server.py` pada direktori utama.
3. **Verifikasi Server Aktif:** Server berhasil berjalan pada <http://127.0.0.1:8088> seperti terlihat pada log terminal. Server ini melayani beberapa protokol sekaligus, termasuk HTTP API untuk Case UTS.
4. **Konfigurasi Postman:** Dibuat Environment *UTS_Local* dengan variable `base_url = http://127.0.0.1:8088`. Header `Content-Type: application/json` ditambahkan di level Collection.



3.2 Request 1: GET /api/users

Tujuan: Mengambil daftar seluruh user yang tersedia di server.

Method & URL: GET <http://127.0.0.1:8088/api/users>

Request Headers: Content-Type: application/json

Request Body: Tidak ada.

Response Status: `200 OK` membuktikan server lokal berhasil merespon request GET. Data yang muncul sesuai dengan data awal yang disediakan aplikasi demo.

PUTRI SAHIRA WULANDARI-UTS-25120500004-comp2 > GET - user

GET http://127.0.0.1:8088/api/users

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body: 200 OK · 34 ms · 919 B

```

1 {
2   "resource": "users",
3   "case": "User",
4   "count": 2,
5   "methods": [
6     "GET",
7     "POST",
8     "PUT",
9     "PATCH",
10    "DELETE"
11  ],
12  "canonicalPath": "/api/users",
13  "itemPathExample": "/api/users/1",
14  "data": [
15    {
16      "id": 1,
17      "username": "student.demo",
18      "email": "student.demo@example.edu",
19      "role": "student",
20      "status": "active"
    }
  ]
}

```

3.3 Request 2: GET /api/users/1

Tujuan: Mengambil detail user dengan id spesifik yaitu 1.

Method & URL: GET /api/users/1

Request Headers: Content-Type: *application/json*

Request Body: Tidak ada.

Response Status*: 200 OK

Response Body: Object JSON berisi detail *'student.demo'*.

Analisis: Penggunaan path parameter *'/1'* berhasil mengambil data spesifik. Ini menunjukkan implementasi routing dinamis di sisi server. Jika id tidak ditemukan, server akan merespon *404 Not Found*.

PUTRI SAHIRA WULANDARI-UTS-25120500004-comp2 > GET - Detail User

GET http://127.0.0.1:8088/api/users/1

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body: 200 OK · 146 ms · 470 B

```

1 {
2   "id": 1,
3   "username": "student.demo",
4   "email": "student.demo@example.edu",
5   "role": "student",
6   "status": "active",
7   "department": "Sains Data"
8 }

```

3.4 Request 3: POST /api/users

Tujuan: Membuat user baru bernama *putri.sahira* di database lokal.

Method & URL: POST <http://127.0.0.1:8088/api/users>

Request Headers: Content-Type: application/json

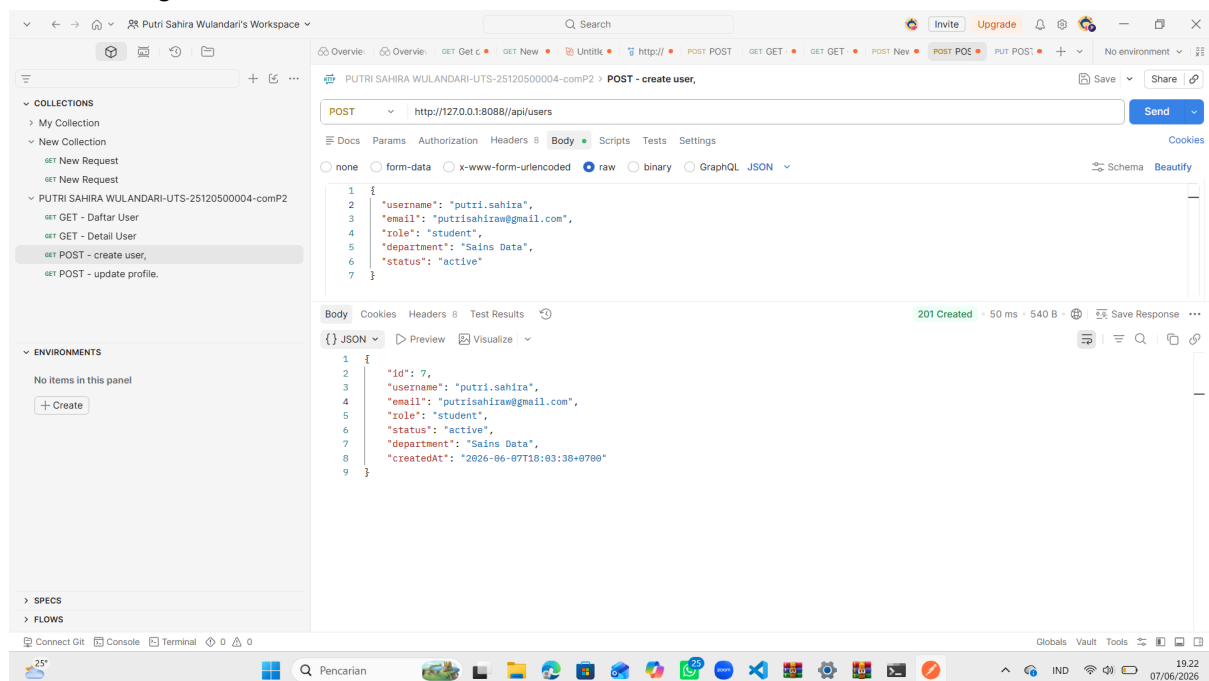
Request Body:

```
{
  "username": "putri.sahira",
  "email": "putrisahiraw@gmail.com",
  "role": "student",
  "department": "Sains Data",
  "status": "active"
}
```

Response Status: `201 Created`

Response Body: Object JSON user baru dengan `id: 7`.

Analisis: Status `201 Created` adalah response yang tepat untuk method POST yang sukses. Server melakukan auto-increment pada `id` dan mengembalikan data yang baru dibuat sebagai konfirmasi.



Error yang Ditemui pada POST:

Tujuan: Membuat user baru pada server lokal.

Method & URL: POST <http://127.0.0.1:8088/api/users>

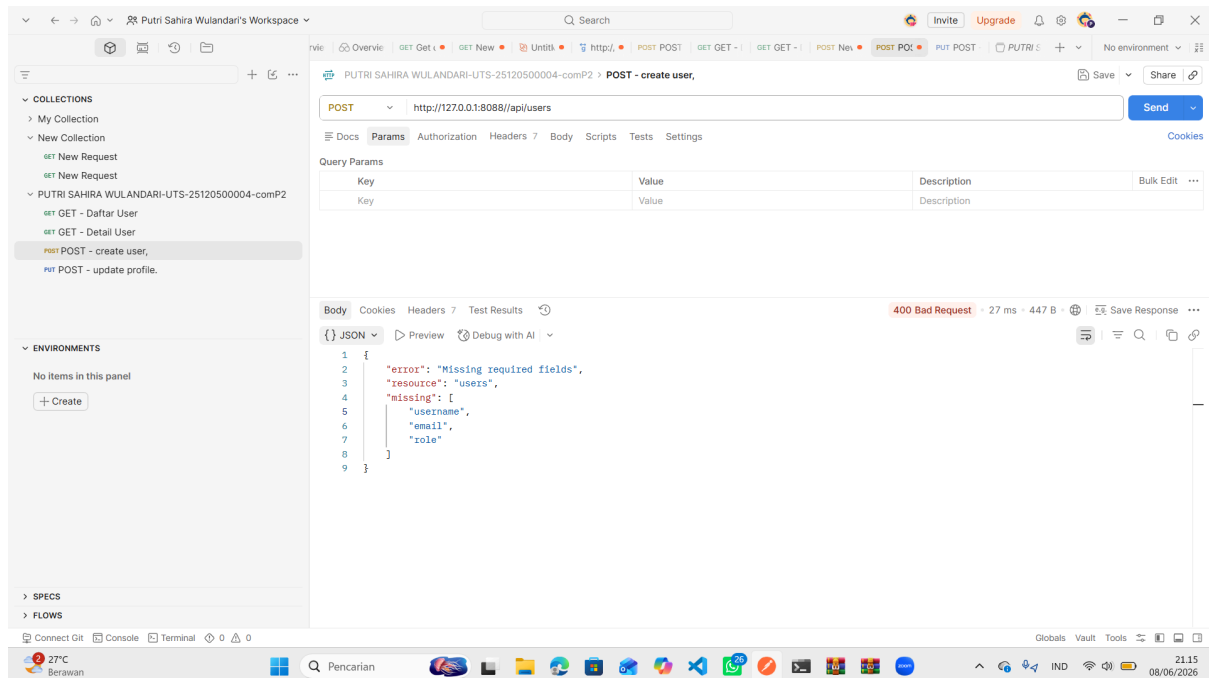
Request Headers: Content-Type: application/json

Skenario 1: Percobaan Gagal - Missing Required Fields

Request Body: Kosong atau tidak lengkap.

Response Status: 400 Bad Request

Analisis : Server lokal menerapkan validasi strict. Field username, email, dan role wajib di kirim.jika tidak ada, server menolak dengan status 400 dan memberitahu field apa yang kurang.



3.5 Request 4: PUT /api/users/1

Tujuan: Mengubah `department` user id 1 menjadi "Sains Data UTS P2".

Method & URL: `PUT /api/users/1`

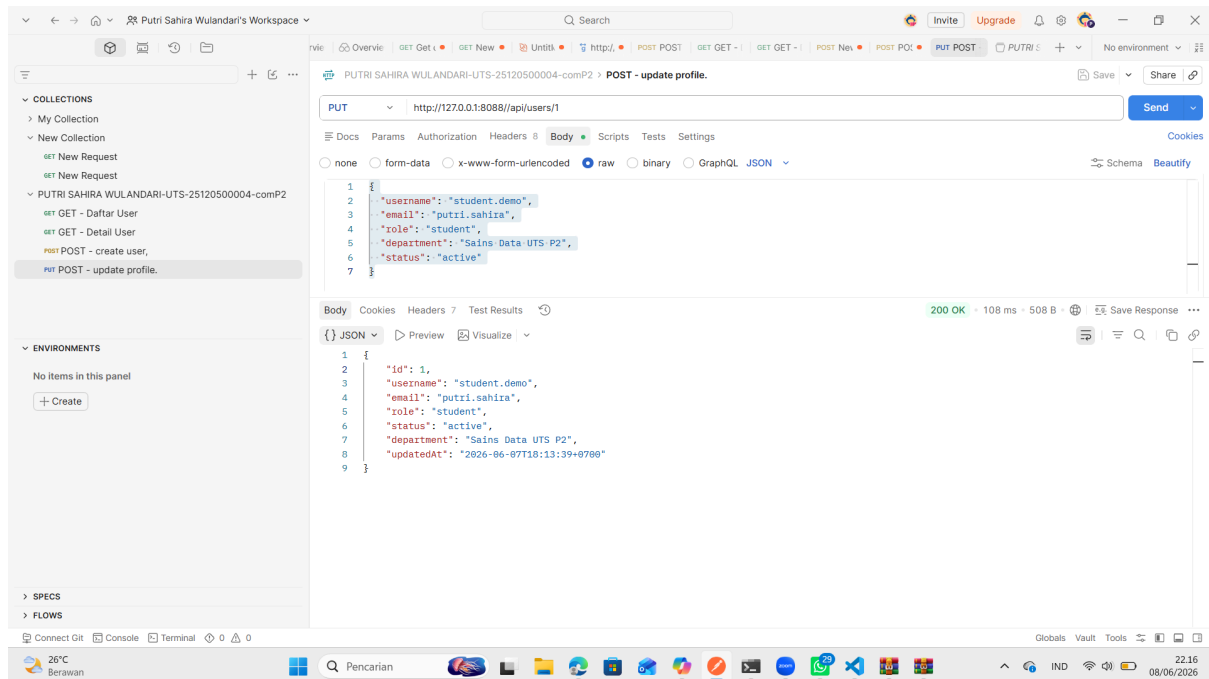
Request Body:

```
{
  "username": "student.demo",
  "email": "putri.sahira",
  "role": "student",
  "department": "Sains Data UTS P2",
  "status": "active"
}
```

Response Status: `200 OK`

Response Body: Object JSON dengan field `department` sudah berubah.

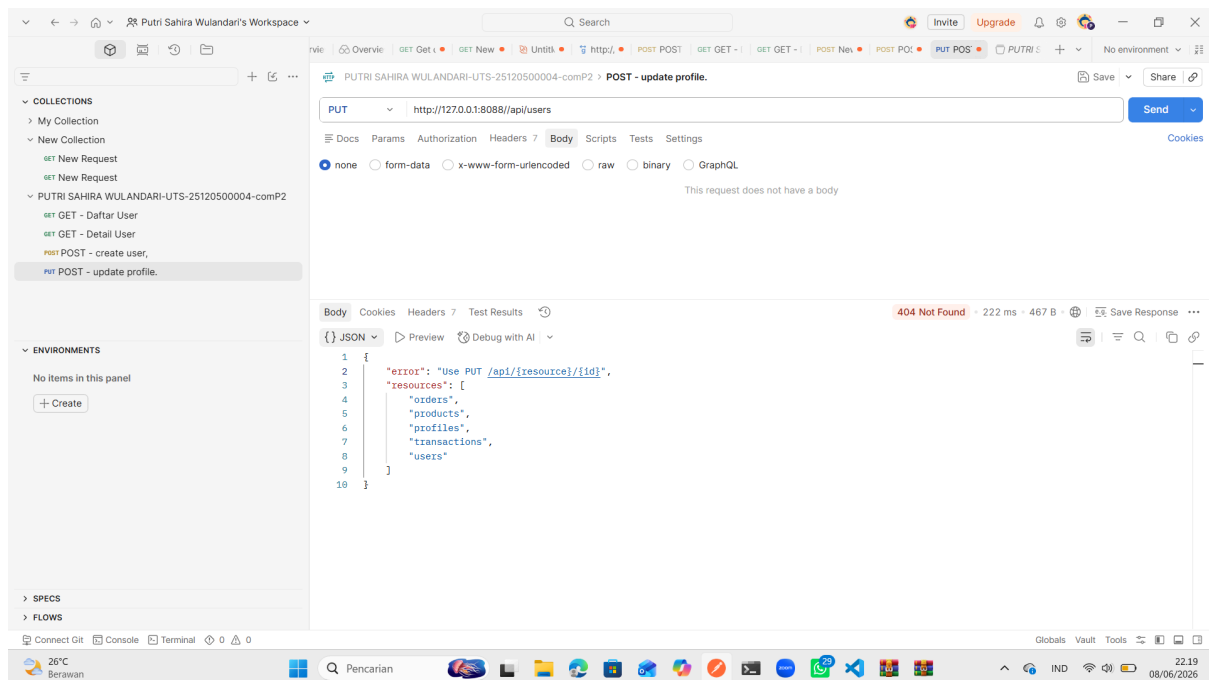
Analisis: Method PUT berhasil melakukan replace data. Status `200 OK` digunakan karena tidak ada resource baru yang dibuat. Percobaan ini membuktikan bahwa API ini tidak mendukung partial update, sehingga seluruh field required harus dikirim ulang.



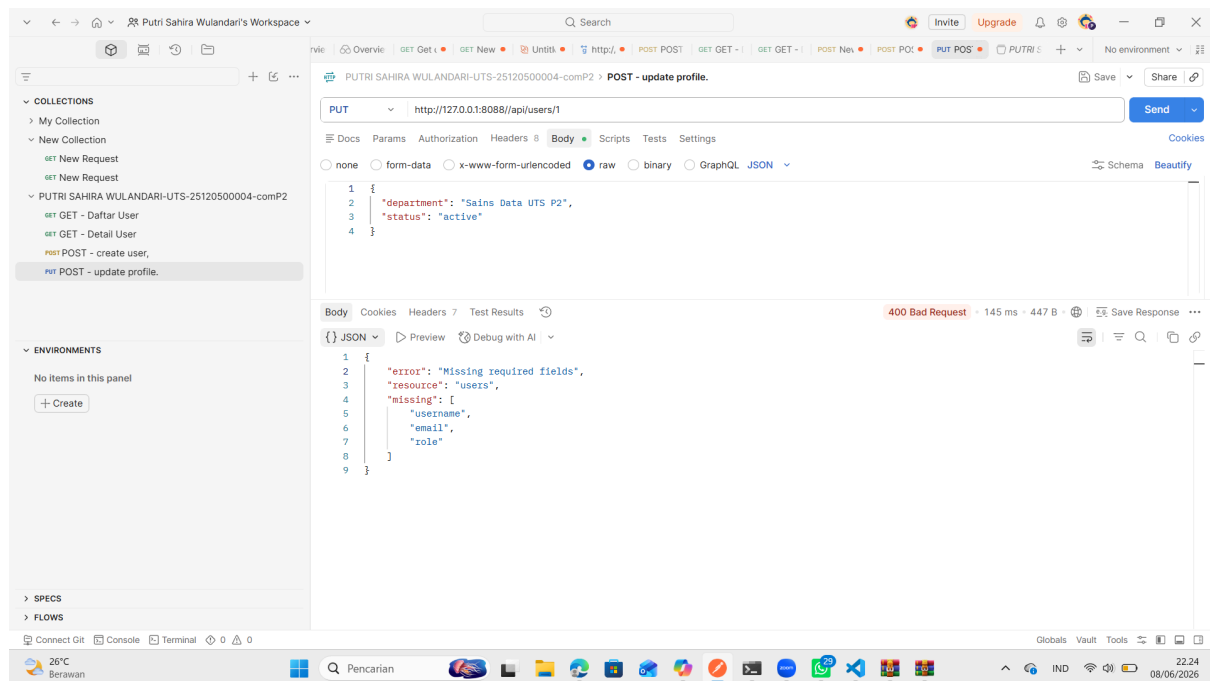
Error yang Ditemui pada PUT:

- Error 404 Not Found:** Terjadi karena URL yang digunakan adalah `/api/users`` tanpa `/1``. Server tidak menemukan resource yang akan di-update.
- Error 400 Bad Request:** Server menolak request karena method PUT pada API ini tidak mendukung partial update. Berbeda dengan PATCH, PUT mewajibkan client mengirimkan seluruh representasi resource, termasuk field yang tidak diubah. Ini membuktikan sifat idempotent dari PUT: request yang sama harus menghasilkan state yang sama.

404 Not Found



400 Bad Request



3.6 Rangkuman Status Code

Status	Request	Penyebab Utama	Tindak Lanjut
200 OK	GET, PUT	Request valid dan diproses	Tidak perlu aksi
201 Created	POST	Data baru berhasil disimpan	Catat id baru
400 Bad Request	POST, PUT	Field tidak sesuai atau kurang	Cek dokumentasi API
404 Not Found	PUT	Path parameter id tidak ada	Perbaiki URL

BAB IV KESIMPULAN DAN SARAN

4.1 Kesimpulan

1. Pengujian Case 1 User/Profile API telah berhasil dilakukan dengan 4 request yaitu 2 GET, 1 POST, dan 1 PUT. Semua request menghasilkan status code `2xx` yang menandakan sukses.
2. Setiap HTTP Method memiliki karakteristik berbeda. GET tidak memerlukan body, POST digunakan untuk create dengan status `201`, dan PUT digunakan untuk update full resource.
3. Status code `4xx` yang ditemui selama praktikum seluruhnya disebabkan oleh kesalahan dari sisi client, seperti field tidak valid dan URL yang salah. Hal ini membuktikan pentingnya validasi input dan membaca pesan error dari server.

4.2 Saran

1. Untuk pengembangan API selanjutnya, disarankan untuk menambahkan dukungan method PATCH agar client dapat melakukan partial update tanpa harus mengirim seluruh object.
2. Dokumentasi API sebaiknya mencantumkan daftar `required fields` dan `allowed fields` secara eksplisit untuk setiap endpoint agar mengurangi trial-error dari pengguna.

4.3 Github

[Link Github](#)

4.4 Wireshark

Menangkap dari Adapter for loopback traffic capture

Berkas Sunting Lihat Go Tangkap Analisis Statistik Telefoni Nirkabel Perangkat Bantuan

Filter: http

No.	Time	Source	Destination	Protocol	Length	Info
40	12.900275500	127.0.0.1	127.0.0.1	HTTP	744	GET / HTTP/1.1
44	13.261196800	127.0.0.1	127.0.0.1	HTTP	27024	HTTP/1.0 200 OK (text/html)
56	13.322015600	127.0.0.1	127.0.0.1	HTTP	627	GET /static/app.css HTTP/1.1
58	13.322454200	127.0.0.1	127.0.0.1	HTTP	612	GET /static/app.js HTTP/1.1
64	13.331633900	127.0.0.1	127.0.0.1	HTTP	18573	HTTP/1.0 200 OK (application/javascript)
65	13.331731500	127.0.0.1	127.0.0.1	HTTP	4662	HTTP/1.0 200 OK (text/css)
82	13.349472900	127.0.0.1	127.0.0.1	HTTP	605	GET /api/health HTTP/1.1
84	13.349973600	127.0.0.1	127.0.0.1	HTTP	607	GET /api/products HTTP/1.1
88	13.352573000	127.0.0.1	127.0.0.1	HTTP/1.1	151	HTTP/1.0 200 OK, JSON (application/json)
96	13.357016900	127.0.0.1	127.0.0.1	HTTP/1.1	686	HTTP/1.0 200 OK, JSON (application/json)
141	26.928316200	127.0.0.1	127.0.0.1	HTTP	255	GET //api/users HTTP/1.1
145	26.945781400	127.0.0.1	127.0.0.1	HTTP/1.1	924	HTTP/1.0 200 OK, JSON (application/json)
166	32.292628300	127.0.0.1	127.0.0.1	HTTP	257	GET //api/users/1 HTTP/1.1
170	32.297225200	127.0.0.1	127.0.0.1	HTTP/1.1	237	HTTP/1.0 200 OK, JSON (application/json)
191	38.051758900	127.0.0.1	127.0.0.1	HTTP/1.1	457	POST //api/users HTTP/1.1, JSON (application/json)
195	38.061337200	127.0.0.1	127.0.0.1	HTTP/1.1	240	HTTP/1.0 201 Created, JSON (application/json)
216	43.847999900	127.0.0.1	127.0.0.1	HTTP/1.1	457	PUT //api/users/1 HTTP/1.1, JSON (application/json)
220	43.889567000	127.0.0.1	127.0.0.1	HTTP/1.1	237	HTTP/1.0 200 OK, JSON (application/json)

> Frame 40: Packet, 744 bytes on wire (5952 bits), 744 bytes captured (5952 bits) on interface \Device\NPF_{...} id 0

> Null/Loopback

> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

> Transmission Control Protocol, Src Port: 64942, Dst Port: 8088, Seq: 1, Ack: 1, Len: 700

> Hypertext Transfer Protocol

0000 02 00 00 00 45 00 02 e4 bc 45 40 00 00 06 3d ccE....E@...2.

0010 7f 00 00 01 7f 00 00 01 fd ae 1f 98 17 f9 bd d5d...Q...Z

0020 4e 69 8c 95 50 18 00 ff e3 2b 00 00 47 45 54 20 ?uSP...BT-GET

0030 2f 20 48 54 54 50 2f 31 2e 31 0d 0a 48 6f 73 74 / HTTP/1.1-Host

0040 3a 20 31 32 37 2e 30 2e 30 2e 31 3a 38 30 38 38 : 127.0.0.1:8088

0050 0d 0a 43 6f 6e 6e 65 63 74 69 6f 6e 3a 20 6b 65 ..Connection: ke

0060 65 70 2d 61 6c 69 76 65 0d 0a 73 65 63 2d 63 68 ep-alive-sec-ch

0070 2d 75 61 3a 20 22 4d 69 63 72 6f 73 6f 66 74 20 -ua: "Microsoft

0080 45 64 67 65 22 3b 76 3d 22 31 34 39 22 2c 20 22 Edge";v="149", "

0090 43 68 72 6f 6d 69 75 6d 22 3b 76 3d 22 31 34 39 Chromium";v="149

00a0 22 2c 20 22 4e 6f 74 29 41 3b 42 72 61 6e 64 22 ", "Not) A;Brand"

00b0 3b 76 3d 22 32 34 22 0d 0a 73 65 63 2d 63 68 2d ;v="24"-sec-ch

00c0 75 61 2d 6d 6f 62 69 6c 65 3a 20 3f 30 0d 0a 73 ua-mobil e: 70-s

00d0 65 63 2d 63 68 2d 75 61 2d 70 6c 61 74 66 6f 72 ec-ch-ua-platfor

00e0 6d 3a 20 22 57 69 6e 64 6f 77 73 22 0d 0a 55 70 m: "Windows".Up

00f0 67 72 61 64 65 2d 49 6e 73 65 63 75 72 65 2d 52 grade-In secure-R

*Adapter for loopback traffic capture

Berkas Sunting Lihat Go Tangkap Analisis Statistik Telefoni Nirkabel Perangkat Bantuan

Filter: http

No.	Time	Source	Destination	Protocol	Length	Info
28	6.627911100	127.0.0.1	127.0.0.1	HTTP	744	GET / HTTP/1.1
32	6.653622900	127.0.0.1	127.0.0.1	HTTP	27024	HTTP/1.0 200 OK (text/html)
44	6.690957200	127.0.0.1	127.0.0.1	HTTP	627	GET /static/app.css HTTP/1.1
46	6.691703900	127.0.0.1	127.0.0.1	HTTP	612	GET /static/app.js HTTP/1.1
50	6.696288100	127.0.0.1	127.0.0.1	HTTP	18573	HTTP/1.0 200 OK (application/javascript)
58	6.697649800	127.0.0.1	127.0.0.1	HTTP	4662	HTTP/1.0 200 OK (text/css)
70	6.709820500	127.0.0.1	127.0.0.1	HTTP	605	GET /api/health HTTP/1.1
72	6.709980600	127.0.0.1	127.0.0.1	HTTP	607	GET /api/products HTTP/1.1
76	6.715713600	127.0.0.1	127.0.0.1	HTTP/1.1	151	HTTP/1.0 200 OK, JSON (application/json)
84	6.722279800	127.0.0.1	127.0.0.1	HTTP/1.1	686	HTTP/1.0 200 OK, JSON (application/json)
131	21.048743100	127.0.0.1	127.0.0.1	HTTP	255	GET //api/users HTTP/1.1
133	21.050013600	127.0.0.1	127.0.0.1	HTTP/1.1	924	HTTP/1.0 200 OK, JSON (application/json)
188	39.121246600	127.0.0.1	127.0.0.1	HTTP	257	GET //api/users/1 HTTP/1.1
192	39.161241600	127.0.0.1	127.0.0.1	HTTP/1.1	237	HTTP/1.0 200 OK, JSON (application/json)
225	45.249470800	127.0.0.1	127.0.0.1	HTTP/1.1	457	POST //api/users HTTP/1.1, JSON (application/json)
229	45.258184400	127.0.0.1	127.0.0.1	HTTP/1.1	240	HTTP/1.0 201 Created, JSON (application/json)
238	49.863369700	127.0.0.1	127.0.0.1	HTTP/1.1	457	PUT //api/users/1 HTTP/1.1, JSON (application/json)
242	49.938062700	127.0.0.1	127.0.0.1	HTTP/1.1	237	HTTP/1.0 200 OK, JSON (application/json)
263	54.704407300	127.0.0.1	127.0.0.1	HTTP	255	GET //api/users HTTP/1.1
267	54.739383600	127.0.0.1	127.0.0.1	HTTP/1.1	924	HTTP/1.0 200 OK, JSON (application/json)

> Frame 28: Packet, 744 bytes on wire (5952 bits), 744 bytes captured (5952 bits) on interface \Device\NPF_{...} id 0

> Null/Loopback

> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

> Transmission Control Protocol, Src Port: 51044, Dst Port: 8088, Seq: 1, Ack: 1, Len: 700

> Hypertext Transfer Protocol

No: 28 - Time: 6.627911100 - Source: 127.0.0.1 - Destination: 127.0.0.1 - Protocol: HTTP - Length: 744 - Info: GET / HTTP/1.1

☒ Tampilkan byte paket Tata letak: Vertical (Stacked)

0000 02 00 00 00 45 00 02 e4 c7 34 40 00 00 06 32 ddE....@...2.

0010 7f 00 00 01 7f 00 00 01 c7 64 1f 98 51 c0 7f 5ad...Q...Z

0020 3f 9f 77 35 50 18 00 ff 42 54 00 00 47 45 54 20 ?uSP...BT-GET

0030 2f 20 48 54 54 50 2f 31 2e 31 0d 0a 48 6f 73 74 / HTTP/1.1-Host

0040 3a 20 31 32 37 2e 30 2e 30 2e 31 3a 38 30 38 38 : 127.0.0.1:8088

0050 0d 0a 43 6f 6e 6e 65 63 74 69 6f 6e 3a 20 6b 65 ..Connection: ke

0060 65 70 2d 61 6c 69 76 65 0d 0a 73 65 63 2d 63 68 ep-alive-sec-ch

0070 2d 75 61 3a 20 22 4d 69 63 72 6f 73 6f 66 74 20 -ua: "Microsoft

0080 45 64 67 65 22 3b 76 3d 22 31 34 39 22 2c 20 22 Edge";v="149", "

0090 43 68 72 6f 6d 69 75 6d 22 3b 76 3d 22 31 34 39 Chromium";v="149

00a0 22 2c 20 22 4e 6f 74 29 41 3b 42 72 61 6e 64 22 ", "Not) A;Brand"

00b0 3b 76 3d 22 32 34 22 0d 0a 73 65 63 2d 63 68 2d ;v="24"-sec-ch

00c0 75 61 2d 6d 6f 62 69 6c 65 3a 20 3f 30 0d 0a 73 ua-mobil e: 70-s

00d0 65 63 2d 63 68 2d 75 61 2d 70 6c 61 74 66 6f 72 ec-ch-ua-platfor

00e0 6d 3a 20 22 57 69 6e 64 6f 77 73 22 0d 0a 55 70 m: "Windows".Up

00f0 67 72 61 64 65 2d 49 6e 73 65 63 75 72 65 2d 52 grade-In secure-R

4.5 PPT Google Slide

[Link](#)

